SFMC Custom Activity Flask アプリケーション 技術ドキュメント

Version 1.0

目次

1. 1. プロジェクト概要
2. 2. API リファレンス
3. 3. モデル定義
4. 4. フロントエンド仕様
5. 5. config.json 仕様書
6. 6. セットアップ・デプロイガイド

SFMC Custom Activity - Flask アプリケーション概要

1. プロジェクト概要

本プロジェクトは、**Salesforce Marketing Cloud (SFMC) Journey Builder** 向けのカスタムアクティビティを提供するWebアプリケーションです。
Flask（Python）をバックエンドとして使用し、Journey Builder のジャーニーフロー内でユーザー情報を **Google Cloud Firestore** に登録する機能を実現しています。

主な機能

- Journey Builder カスタムアクティビティとしての動作（設定UI + REST APIエンドポイント）
- ジャーニー実行時にコンタクト情報を Firestore へ保存
- DialogOne との連携を目的としたデータ受け渡し

技術スタック

カテゴリ	技術
バックエンド	Python 3 / Flask 3.0.0
WSGI サーバー	Gunicorn 19.9.0
データベース	Google Cloud Firestore
ロギング	Google Cloud Logging
フロントエンド	HTML / JavaScript / Semantic UI
JB連携ライブラリ	Postmonger.js 0.0.14
コンテナ	Docker
デプロイ先	Google Cloud Run

2. ディレクトリ構成

```
sfmc-ac-flask/
├── app.py                  # Flask メインアプリケーション
├── config.json             # SFMC カスタムアクティビティ定義ファイル
├── Dockerfile              # Docker コンテナビルド定義
├── Procfile                # Heroku / Cloud Run 用プロセス定義
├── requirements.txt        # Python 依存パッケージ一覧
├── README.md               # プロジェクト概要（簡易）
├── models/
|   └── users.py            # Users モデル（Firestore データ操作）
├── static/
|   ├── css/
|   |   ├── main.css        # カスタムスタイル
|   |   ├── semantic.css    # Semantic UI（フル版）
|   |   └── semantic.min.css # Semantic UI（圧縮版）
|   ├── img/
|   |   └── done.png        # アクティビティアイコン（推定）
|   └── js/
|       ├── index.js        # カスタムアクティビティ UI ロジック
|       ├── postmonger.js   # JB 通信ライブラリ
|       ├── semantic.js     # Semantic UI JS（フル版）
|       └── semantic.min.js # Semantic UI JS（圧縮版）
├── templates/
|   └── index.html          # カスタムアクティビティ設定画面テンプレート
└── docs/                   # ドキュメント（本ディレクトリ）
```

3. アプリケーションの全体フロー

```
┌──────────────────────────────────────────────────────────┐
|              SFMC Journey Builder                         |
|                                                           |
|   1. ジャーニーにカスタムアクティビティを配置            |
|   2. 設定画面（index.html）で content_id を入力          |
|   3. ジャーニー実行時に /execute エンドポイントへPOST     |
|                                                           |
└──────────────────────┬───────────────────────────────────┘
                       |
                       ▼
┌──────────────────────────────────────────────────────────┐
|              Flask アプリケーション（app.py）             |
|                                                           |
|   /execute (POST)                                         |
|     → inArguments からユーザーデータを取得                |
|     → Users モデルで Firestore へ保存                     |
```

```
│                                                          │
│   /save, /publish, /validate (POST)                      │
│     → JB ライフサイクルイベントの受信                    │
│                                                          │
└──────────────────────┬──────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────┐
│              Google Cloud Firestore                      │
│                                                          │
│   コレクション: smc_connect_users                        │
│   フィールド: contact_key, uid, acid, campaign_id,       │
│              content_id, send_flg                        │
└──────────────────────────────────────────────────────────┘
```

---

4. 連携先サービス

Salesforce Marketing Cloud (SFMC)

- Journey Builder のカスタムアクティビティとして動作
- `config.json` で定義されたエンドポイントURLを通じてJBと通信

Google Cloud Firestore

- ジャーニー実行時のユーザーデータを永続化
- コレクション名: `smc_connect_users`

Google Cloud Logging

- アプリケーションログをCloud Loggingへ送信
- ログレベル: DEBUG

DialogOne

- `config.json` の `lang` 設定で "DialogOne" と記載
- SFMC のデータエクステンション `DONE_USERS` を参照してユーザー属性を取得

API リファレンス

本ドキュメントでは、Flask アプリケーションが提供する全 REST API エンドポイントについて記載します。

エンドポイント一覧

メソッド	パス	用途	カテゴリ
GET	`/`	設定画面の表示	UI
GET	`/index.html`	設定画面の表示（別名）	UI
GET	`/config.json`	アクティビティ定義の取得	JB連携
POST	`/execute`	ジャーニー実行時の処理	JB連携
POST	`/publish`	ジャーニー公開時の処理	JB連携
POST	`/save`	アクティビティ保存時の処理	JB連携
POST	`/validate`	アクティビティ検証時の処理	JB連携
POST	`/dotest`	テスト用データ受信	テスト
GET	`/storetest`	Firestore テストデータ挿入	テスト

エンドポイント詳細

GET `/` • `/index.html`

概要: Journey Builder のカスタムアクティビティ設定画面を返します。

レスポンス: `templates/index.html` をレンダリングした HTML

説明:
Journey Builder でアクティビティを選択した際に表示される設定モーダル画面です。ユーザーは `content_id`（コンテンツID）を入力できます。

GET `/config.json`

概要: SFMC カスタムアクティビティの定義ファイルを返します。

レスポンス: JSON（`config.json` の内容）

レスポンス例:

```
{
  "workflowApiVersion": "1.1",
  "metaData": {
    "icon": "/static/img/done.png",
    "category": "message"
  },
  "type": "REST",
  "lang": {
    "en-US": {
      "name": "DialogOne",
      "description": "Connet to DialogOne"
    }
  },
  "arguments": {
    "execute": {
      "inArguments": [],
      "outArguments": [],
      "timeout": 20000,
      "retryCount": 1,
      "retryDelay": 10000,
      "concurrentRequests": 1,
      "url": "https://sfmc-done-test-akb3xrrmgq-uc.a.run.app/execute"
    }
  },
  "configurationArguments": {
    "save": { "url": "..." },
    "publish": { "url": "..." },
    "validate": { "url": "..." },
    "stop": { "url": "..." }
  },
  "userInterfaces": {
    "configModal": {
      "height": 200,
      "width": 300,
      "fullscreen": true
    }
  }
}
```

主要フィールド説明:

フィールド	説明
`workflowApiVersion`	JB Workflow API のバージョン（1.1）
`metaData.icon`	アクティビティのアイコン画像パス
`metaData.category`	アクティビティのカテゴリ（ `message ）`
`type`	アクティビティタイプ（ `REST ）`
`arguments.execute`	ジャーニー実行時の設定（URL、タイムアウト、リトライ等）
`configurationArguments`	各ライフサイクルイベント（save/publish/validate/stop）のURL
`userInterfaces.configModal`	設定モーダルのサイズ設定

POST `/execute`

概要: ジャーニー実行時に呼び出され、ユーザーデータを Firestore に保存します。

リクエストボディ:

```
{
  "inArguments": [
    {
      "contact_key": "user@example.com",
      "uid": "12345",
      "acid": "50dbb3948b683b5a",
      "campaign_id": "campaign_001",
      "content_id": "39812"
    }
  ]
}
```

inArguments フィールド:

フィールド	型	説明
`contact_key`	string	SFMC コンタクトキー（ `{{Contact.Key}} ）`
`uid`	string	ユーザーID（データエクステンション `DONE_USERS` の `UID ）`
`acid`	string	ACID（データエクステンション `DONE_USERS` の `ACID ）`
`campaign_id`	string	キャンペーンID（データエクステンション `DONE_USERS` の `campaign_id ）`

フィールド	型	説明
`content_id`	string	コンテンツID（設定画面で入力された値）

レスポンス: `200 OK` （ `Success` ）

処理内容:
1. リクエストボディから `inArguments[0]` を取得
2. `Users` モデルを生成し、Firestore コレクション `smc_connect_users` にドキュメントを追加
3. 保存されるデータには `send_flg: false` が自動付与される

POST `/publish`

概要: ジャーニーが公開（アクティベート）された際に呼び出されます。

レスポンス: `200 OK` （ `Success` ）

処理内容: ログ出力のみ。追加の処理は実装されていません。

POST `/save`

概要: アクティビティの設定が保存された際に呼び出されます。

レスポンス: `index.html` テンプレートのレンダリング結果

処理内容: ログ出力のみ。

POST `/validate`

概要: ジャーニー公開前のバリデーション時に呼び出されます。

レスポンス: `200 OK` （ `Success` ）

処理内容: ログ出力のみ。常に成功を返します。

POST `/dotest`

概要: テスト用エンドポイント。JSON ボディから `uid` を受け取り、コンソールに出力します。

リクエストボディ:

```
{
  "uid": "12345"
}
```

レスポンス: `200 OK` （ `Success` ）

GET `/storetest`

概要: Firestore へテストデータを挿入するためのエンドポイントです。

レスポンス: `200 OK` （ `Success` ）

処理内容: `test_collection` コレクションに以下の固定データを挿入します。

```
{
  "contact_key": "ryo-tsubota@dac.co.jp",
  "uid": "1212",
  "acid": "50dbb3948b683b5a",
  "content_id": "39812",
  "send_flg": false
}
```

> **注意**: このエンドポイントは開発・テスト目的のものであり、本番環境では無効化することを推奨します。

モデル定義

Users モデル

ファイル: `models/users.py`

概要

Firestore へのユーザーデータ登録を担当するモデルクラスです。
SFMC Journey Builder の `/execute` エンドポイントから呼び出され、ジャーニー実行時のコンタクト情報を永続化します。

クラス定義

```
class Users:
    def __init__(self, db, jdata)
    def insert(self, collection_name)
```

コンストラクタ

```
def __init__(self, db, jdata)
```

引数	型	説明
`db`	`firestore.Client`	Firestore クライアントインスタンス
`jdata`	`dict`	SFMC から受信した inArguments データ

コンストラクタでは、受信データから以下のフィールドを抽出して `self.data` に格納します：

フィールド	説明	ソース
`contact_key`	SFMC コンタクトキー	`jdata['contact_key']`
`uid`	ユーザーID	`jdata['uid']`
`acid`	ACID	`jdata['acid']`
`campaign_id`	キャンペーンID	`jdata['campaign_id']`
`content_id`	コンテンツID	`jdata['content_id']`

フィールド	説明	ソース
`send_flg`	送信フラグ（初期値: `False`）	固定値

メソッド

`insert(collection_name)`

Firestore の指定コレクションにドキュメントを追加します。

引数	型	説明
`collection_name`	`str`	保存先の Firestore コレクション名

使用例（`app.py` 内）:

```
users = Users(db, jdata)
users.insert("smc_connect_users")
```

例外処理: Firestore への書き込みに失敗した場合、エラーをログ出力した後に例外を再送出します。

Firestore データ構造

コレクション名: `smc_connect_users`

フィールド	型	説明
`contact_key`	string	SFMC コンタクトキー（メールアドレス等）
`uid`	string	ユーザーID
`acid`	string	ACID
`campaign_id`	string	キャンペーンID
`content_id`	string	コンテンツID
`send_flg`	boolean	送信済みフラグ（初期値: `false`）

> **備考**: `send_flg` は初期値 `false` で挿入され、後続の処理（外部システム等）で `true` に更新される設計と推定されます。

未実装機能

- バリデーションチェック（`validation_data` メソッド）がコメントアウトされており、未実装の状態です。

フロントエンド仕様

概要

カスタムアクティビティの設定画面（UI）は、Journey Builder の iframe 内で表示されます。Postmonger.js を使用して Journey Builder 本体と通信し、アクティビティの設定情報をやり取りします。

設定画面（index.html）

画面構成

要素	説明
Content_id 入力フィールド	コンテンツIDをユーザーが入力するテキストフィールド
Save ボタン	設定を保存し、Journey Builder に反映する
Cancel ボタン	変更を破棄して設定画面を閉じる

使用ライブラリ

ライブラリ	バージョン	用途
jQuery	3.7.1 (slim)	DOM操作
Semantic UI	-	UIフレームワーク（フォーム、ボタン等）
Postmonger.js	0.0.14	Journey Builder との通信

スタイル（main.css）

クラス	説明
`.container`	最大幅960px、中央揃えのメインコンテナ
`.btn_group`	ボタングループ（上マージン30px）
`.topmargin`	上マージン30px

クラス	説明
`.center`	テキスト中央揃え

JavaScript ロジック（index.js）

定数

変数	値	説明
`connection`	`Postmonger.Session`	JB との通信セッション
`activity`	`null`（初期値）	アクティビティオブジェクト
`dataExtentionName`	`"DONE_USERS"`	参照するデータエクステンション名

イベントフロー

```
DOMContentLoaded
    |
    ├── setupEventHandlers()   // ボタンイベントの設定
    ├── connection.on('initActivity', onInitActivity)   // 初期化イベントのバインド
    └── connection.trigger('ready')   // JB に準備完了を通知
          |
          ▼
    onInitActivity(payload)
          |  ← JB からアクティビティオブジェクトを受信
          └── activity 変数にペイロードを保存
```

関数詳細

`onInitActivity(payload)`

Journey Builder から受信したアクティビティ情報を処理します。

- `activity` 変数にペイロードを格納
- `inArguments` の有無を確認
- デバッグ用にコンソールへ情報を出力

`onDoneButtonClick()`

Save ボタンクリック時の処理です。

処理フロー:

1. `activity.metaData.isConfigured = true` を設定（アクティビティが設定済みであることをJBに通知）
2. 入力された `content_id` を取得
3. `inArguments` を構築:

```
activity.arguments.execute.inArguments = [{
    content_id: content_id,                                  // 画面入力値
    contact_key: "{{Contact.Key}}",                          // SFMC コンタクトキー
    uid: "{{Contact.Attribute.DONE_USERS.UID}}",             // データエクステンション参照
    acid: "{{Contact.Attribute.DONE_USERS.ACID}}",           // データエクステンション参照
    name: "{{Contact.Attribute.DONE_USERS.name}}",           // データエクステンション参照
    campaign_id: "{{Contact.Attribute.DONE_USERS.campaign_id}}" // データエクステンション参照
}];
```

1. アクティビティ名を `content_id` の値に設定
2. `connection.trigger('updateActivity', activity)` で JB にアクティビティ更新を通知

`onCancelButtonClick()`

Cancel ボタンクリック時の処理です。

1. `setActivityDirtyState` を `false` に設定（変更なし扱い）
2. `requestInspectorClose` でインスペクター（設定画面）を閉じる

SFMC データエクステンション参照

フロントエンドでは、SFMC のパーソナライゼーション文字列を使用してデータエクステンション `DONE_USERS` から以下の属性を参照しています：

パーソナライゼーション文字列	参照先
`{{Contact.Key}}`	コンタクトキー
`{{Contact.Attribute.DONE_USERS.UID}}`	ユーザーID
`{{Contact.Attribute.DONE_USERS.ACID}}`	ACID
`{{Contact.Attribute.DONE_USERS.name}}`	名前
`{{Contact.Attribute.DONE_USERS.campaign_id}}`	キャンペーンID

> **注意**: `dataExtentionName` 変数（`"DONE_USERS"`）を変更することで、参照先のデータエクステンションを切り替えられます。コード内に `TODO: データエクステンション名に応じ変更する` のコメントがあります。

config.json 仕様書

概要

`config.json` は、SFMC Journey Builder がカスタムアクティビティを認識・制御するための**アクティビティ定義ファイル**です。
Journey Builder はこのファイルを読み込み、アクティビティの表示名、アイコン、各ライフサイクルイベントのエンドポイントURL等を取得します。

構造

トップレベルプロパティ

プロパティ	値	説明
`workflowApiVersion`	`"1.1"`	Journey Builder Workflow API のバージョン
`type`	`"REST"`	アクティビティタイプ（REST API ベース）

metaData

アクティビティのメタ情報を定義します。

プロパティ	値	説明
`icon`	`"/static/img/done.png"`	JB キャンバス上に表示されるアイコン画像のパス
`category`	`"message"`	アクティビティのカテゴリ（JB のパレットでの表示位置）

lang

アクティビティの多言語表示設定です。

ロケール	name	description
`en-US`	DialogOne	Connet to DialogOne

arguments.execute

ジャーニー実行時の設定です。

プロパティ	値	説明
`inArguments`	`[]`	実行時に渡される引数（フロントエンドで動的に設定）
`outArguments`	`[]`	実行後に返される引数（未使用）
`timeout`	`20000`	タイムアウト（ミリ秒）：20秒
`retryCount`	`1`	リトライ回数
`retryDelay`	`10000`	リトライ間隔（ミリ秒）：10秒
`concurrentRequests`	`1`	同時リクエスト数
`url`	`"https://sfmc-done-test-akb3xrrmgq-uc.a.run.app/execute"`	実行エンドポイントURL

configurationArguments

各ライフサイクルイベントのエンドポイントURLです。

イベント	パス	説明
`save`	`/save`	アクティビティ設定の保存時
`publish`	`/publish`	ジャーニーのアクティベート時
`validate`	`/validate`	ジャーニー公開前のバリデーション時
`stop`	`/stop`	ジャーニー停止時（※エンドポイント未実装）

> **注意**: `/stop` エンドポイントは `config.json` で定義されていますが、`app.py` には対応するルートが実装されていません。

userInterfaces.configModal

アクティビティ設定画面のモーダル設定です。

プロパティ	値	説明
`height`	`200`	モーダルの高さ（px）
`width`	`300`	モーダルの幅（px）
`fullscreen`	`true`	フルスクリーン表示（`true` の場合、width/height は無視される）

Journey Builder ライフサイクル

```
アクティビティ配置
    |
    ▼
設定画面表示（/index.html）
    |
    ▼ ユーザーが設定を保存
/save（POST）
    |
    ▼ ジャーニーを公開
/validate（POST）→ /publish（POST）
    |
    ▼ ジャーニー実行（各コンタクトに対して）
/execute（POST）
    |
    ▼ ジャーニー停止
/stop（POST）　※未実装
```

セットアップ・デプロイガイド

前提条件

- Python 3.x
- Google Cloud プロジェクト（Firestore、Cloud Logging が有効化済み）
- Google Cloud SDK（ローカル開発時）
- Docker（コンテナビルド時）
- SFMC 管理者権限（カスタムアクティビティの登録）

依存パッケージ

`requirements.txt` に定義されたパッケージ一覧：

パッケージ	バージョン	用途
flask	3.0.0	Web フレームワーク
gunicorn	19.9.0	WSGI HTTP サーバー
requests	2.31.0	HTTP クライアント
firebase-admin	6.2.0	Firebase Admin SDK
google-cloud-firestore	2.13.0	Firestore クライアント
google-cloud-logging	3.8.0	Cloud Logging クライアント
PyYAML	6.0.1	YAML パーサー

ローカル開発

1. 環境構築

```
# 仮想環境の作成・有効化
python3 -m venv venv
source venv/bin/activate
```

```
# 依存パッケージのインストール
pip install -r requirements.txt
```

2. Google Cloud 認証

```
# サービスアカウントキーを使用する場合
export GOOGLE_APPLICATION_CREDENTIALS="path/to/service-account-key.json"

# gcloud CLI でログインする場合
gcloud auth application-default login
```

3. アプリケーション起動

```
python app.py
```

アプリケーションは `http://0.0.0.0:8080` で起動します。

Docker ビルド・実行

Dockerfile の構成

```
FROM python:3          # Python 3 ベースイメージ
USER root              # root ユーザーで実行

EXPOSE 8080:8080       # ポート 8080 を公開

# システム・pip の更新
RUN apt-get update
RUN pip install --upgrade pip
RUN pip install --upgrade setuptools

# 依存パッケージのインストール
COPY "./requirements.txt" /tmp
RUN pip install -r /tmp/requirements.txt

# アプリケーションコードのコピー
COPY ./. /app/
WORKDIR "/app"
CMD ["python","app.py"]
```

ビルドと実行

```
# イメージのビルド
docker build -t sfmc-ac-flask .

# コンテナの実行
docker run -p 8080:8080 sfmc-ac-flask
```

Google Cloud Run へのデプロイ

1. Container Registry / Artifact Registry へのプッシュ

```
# プロジェクトIDを設定
export PROJECT_ID=your-gcp-project-id

# イメージのビルドとプッシュ
gcloud builds submit --tag gcr.io/$PROJECT_ID/sfmc-ac-flask

# または Artifact Registry を使用
gcloud builds submit --tag us-central1-docker.pkg.dev/$PROJECT_ID/repo-name/sfmc-ac-flask
```

2. Cloud Run へのデプロイ

```
gcloud run deploy sfmc-ac-flask \
  --image gcr.io/$PROJECT_ID/sfmc-ac-flask \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --port 8080
```

3. デプロイ後の設定

デプロイ後に取得した Cloud Run の URL を `config.json` の各エンドポイントURLに反映してください。

現在の設定値:

```
https://sfmc-done-test-akb3xrrmgq-uc.a.run.app
```

変更が必要な箇所（`config.json`）：
- `arguments.execute.url`
- `configurationArguments.save.url`
- `configurationArguments.publish.url`
- `configurationArguments.validate.url`
- `configurationArguments.stop.url`

Heroku デプロイ（Procfile）

`Procfile` が定義されているため、Heroku へのデプロイも可能です。

```
web: gunicorn app:app --log-file ./log.txt
```

```
heroku create sfmc-ac-flask
git push heroku main
```

SFMC への登録

カスタムアクティビティを SFMC に登録するには、以下の手順が必要です：

1. SFMC の **Package Manager** でアプリケーションパッケージを作成
2. **Journey Builder Activity** コンポーネントを追加
3. エンドポイント URL にデプロイ先の URL を設定
4. `config.json` のURLが正しくデプロイ先を指していることを確認

環境変数

現在のアプリケーションでは明示的な環境変数は使用されていませんが、以下の暗黙の設定が必要です：

設定	説明
Google Cloud 認証情報	Firestore・Cloud Logging アクセスに必要。Cloud Run では自動的にサービスアカウントが適用される
	デフォルトの Firestore データベースを使用

設定	説明
Firestore データベース	